

Name: Xilin Wang, Xiangzhou Tan, Guyi Jia, Cloris He

Professor: Nikolaos Machairas

Course: APANPS5310_D01_2026_2 - SQL

Date: 08/14/2026

Final Project Report – Group 4

1. Introduction and Client-Consulting Scenario

1.1 Client-Consulting Project Scenario

ABC Foodmart is a local grocery chain with two stores in Queens, New York, and plans to open three new stores in Brooklyn. Its staffing, inventory, purchasing, sales, and accounting records are currently managed through separate spreadsheets and paper documents. Our consulting team was hired to design a centralized database and reporting system that can better support the company's expansion.

1.2 Project Motivation and Initial Research

We selected grocery retail because its operations are closely connected: sales affect inventory, inventory drives purchasing, and staffing and operating costs influence store performance. Our review of these business processes showed that ABC Foodmart needed not only centralized data storage, but also a system that connects information across departments for faster and more consistent decision-making.

1.3 Proposed Solution and Business Benefits

We proposed a PostgreSQL relational database supported by a Python ETL pipeline and Metabase dashboards. Together, these components centralize and validate operational data, support cross-functional SQL analysis, and allow management to monitor sales, inventory, workforce, and costs. The solution reduces fragmented record keeping and provides a scalable information system for ABC Food Mart's planned expansion.

2. Team contract

The team established a working agreement with clear responsibilities based on each member's primary role and strengths. The agreed division of responsibilities is summarized in Table below:

Team Member	Primary Role	Major Responsibilities
Xilin Wang	Organizer & Final Integration	Weekly meetings, meeting notes, task assigning, feedback, final revisions, data-generation rules/code, report writing, final PPT & presentation script
Xiangzhou Tan	Database & Dashboard Lead	Initial database schema, technical database development, Metabase dashboard, dashboard deployment/publication
Oracle	Report Lead	Final report Writing, continuous documentation of schema/data/ETL/generation-rule changes
Cloris	Data Validation & Schema Support	Data correction and validation, refinement of synthetic data rules, addressing weaknesses in initial generated data, schema development support

2.1 Teamwork Timeline

The project was completed through five major stages aligned with the weekly checkpoints. Week 1 focused on selecting the ABC Food Mart scenario and defining the initial project direction. Week 2 established business requirements, team responsibilities, and internal deadlines. Week 3 focused on developing the ERD and the 19-table relational schema. Week 4 covered synthetic data generation, ETL, validation, and correction. The final stage focused on dashboard development, report integration, and presentation preparation.

The most challenging stages were database design and data development. Designing the schema required translating business processes into normalized relational structures with appropriate constraints and relationships. Data development required multiple rounds of validation because the initial synthetic dataset contained inconsistencies that had to be corrected before it could reliably support the dashboard and final analysis.

3. Data Selection and Sample

Because actual operational data from ABC Foodmart was not available, a synthetic dataset was generated specifically for this project. The dataset covers all 19 tables in the relational database and was designed to provide sufficient data for database implementation, SQL analysis, and dashboard development without using confidential company information.

The data was not generated as independent random records. Business relationships and constraints were incorporated to maintain consistency across tables.

For example, the following sample data, employee records, were assigned valid stores and job roles, while subsequent staffing data considered employment status, hire dates, approved time off, store operating hours, and weekly working-hour limits. A fixed random seed was used to make the dataset reproducible.

employee_id	store_id	role_id	first_name	last_name	email	hire_date	hourly_pay_rate	employment_status
1	1	1	Emily	Chen	emily.chen1@abcfoodmart.com	2023-03-19	30.09	active
2	2	1	Jason	Lee	jason.lee2@abcfoodmart.com	2024-02-17	36.25	active
3	3	1	Maria	Garcia	maria.garcia3@abcfoodmart.com	2024-04-18	35.19	active
4	3	2	Noah	Martinez	noah.martinez4@abcfoodmart.com	2024-08-23	29.77	active
5	3	6	Olivia	Scott	olivia.scott5@abcfoodmart.com	2023-03-08	22.47	active
6	1	4	Henry	Baker	henry.baker6@abcfoodmart.com	2024-04-20	20.87	active
7	2	3	David	Baker	david.baker7@abcfoodmart.com	2021-08-14	18.13	active
8	3	6	Lucas	Scott	lucas.scott8@abcfoodmart.com	2021-03-09	21.01	active
9	1	6	David	King	david.king9@abcfoodmart.com	2021-01-28	19.16	terminated
10	2	2	Noah	Nelson	noah.nelson10@abcfoodmart.com	2026-03-15	28.66	active
11	3	6	Michael	Scott	michael.scott11@abcfoodmart.com	2024-02-02	20.78	active
12	1	3	Nina	Scott	nina.scott12@abcfoodmart.com	2025-10-11	21.87	active
13	2	6	Olivia	Mitchell	olivia.mitchell13@abcfoodmart.com	2025-08-13	22.47	active
14	1	4	Chloe	Kim	chloe.kim14@abcfoodmart.com	2021-07-28	19.13	active
15	3	4	Emily	Adams	emily.adams15@abcfoodmart.com	2025-11-07	21.05	active
16	2	2	Noah	Wong	noah.wong16@abcfoodmart.com	2025-07-02	28.19	active
17	1	3	Sophia	Mitchell	sophia.mitchell17@abcfoodmart.com	2024-12-20	20.26	active
18	1	3	Michael	Mitchell	michael.mitchell18@abcfoodmart.com	2022-02-25	17.27	active
19	2	3	Liam	Chen	liam.chen19@abcfoodmart.com	2021-03-05	19.53	active
20	3	3	Aisha	Garcia	aisha.garcia20@abcfoodmart.com	2024-05-16	18.53	active

The complete synthetic dataset for all 19 tables is available at: [\[Dataset link\]](#).

4. Database Schema Design

4.1 Business and Database Requirements

The database was designed to integrate five major operational areas of ABC Food Mart: store and workforce management, product and inventory management, vendor and purchasing management, sales and returns, and operating expenses. The

system therefore needed to maintain consistent master data, preserve relationships between operational records, track transaction and inventory history, and enforce business rules such as valid quantities, dates, monetary values, and inventory balances.

4.2 Entity-Relationship Diagram

The ERD diagram [[Lucidchart link](#)] presents the final Entity-Relationship Diagram (ERD), consisting of 19 relational tables organized into five connected business areas:

- **Store and workforce:**
[stores](#), [job_roles](#), [employees](#), [staff_shifts](#), [time_off_requests](#)
- **Products and inventory:**
[product_categories](#), [products](#), [store_inventory](#), [inventory_adjustments](#)
- **Vendors and purchasing:**
[vendors](#), [vendor_products](#), [purchase_orders](#), [purchase_order_items](#), [deliveries](#)
- **Sales and returns:**
[sales_transactions](#), [sales_items](#), [customer_returns](#), [payments](#)
- **Expenses:** [operating_expenses](#)

4.3 Relational Database Design

The 19-table relational schema was designed to reduce duplication while preserving the relationships required by ABC Food Mart. Reusable master data is stored separately and referenced through foreign keys. For example, [job_roles](#) is separated from [employees](#), and [product_categories](#) is separated from [products](#), so role and category information does not need to be repeated across individual records.

Transaction processes use a header-item structure. [sales_transactions](#) stores information applying to the overall sale, while [sales_items](#) stores the individual products within that sale. Similarly, [purchase_orders](#) stores order-level information, while [purchase_order_items](#) records individual products and quantities. This prevents transaction-level attributes such as store, vendor, date, and status from being repeated for every item.

Relationship-specific information is stored in associative tables. [store_inventory](#) represents each store-product combination because inventory levels differ by location, while [vendor_products](#) represents each vendor-product combination because purchase cost, minimum order quantity, and lead time vary by

supplier. The design also separates current state from historical activity: `store_inventory` stores the current balance, while `inventory_adjustments` records the movements that caused inventory to change. Together, these design choices reduce redundancy and support normalization through Third Normal Form (3NF).

4.4 Data Integrity and Business Rules

Data integrity is enforced directly in PostgreSQL using primary keys, foreign keys, `NOT NULL`, `UNIQUE`, `DEFAULT`, and `CHECK` constraints. These constraints enforce rules within and between records, while triggers are used for business rules that require operations across multiple tables.

4.4.1 Entity and Referential Integrity

Primary keys uniquely identify records, while foreign keys ensure that records reference existing entities. The `employees` table provides a simple example:

```
CREATE TABLE employees (  
  employee_id      BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  store_id         BIGINT NOT NULL REFERENCES stores(store_id),  
  role_id          BIGINT NOT NULL REFERENCES job_roles(role_id),  
  first_name       VARCHAR(80) NOT NULL,  
  last_name        VARCHAR(80) NOT NULL,  
  email            VARCHAR(160) UNIQUE,  
  hire_date        DATE NOT NULL,  
  hourly_pay_rate  NUMERIC(8,2) NOT NULL CHECK (hourly_pay_rate >= 0),  
  employment_status VARCHAR(20) NOT NULL DEFAULT 'active'  
    CHECK (employment_status IN ('active', 'on_leave', 'terminated'))  
);
```

The identity primary key provides a unique identifier for each employee, while the two foreign keys ensure that every employee belongs to an existing store and job role. `NOT NULL` makes these relationships mandatory. The same PK/FK principle is applied throughout the schema to connect products, inventory, vendors, purchasing, sales, returns, payments, and expenses.

Relationship tables also use keys to prevent duplicate relationships. For example, `store_inventory` uses:

```
CREATE TABLE store_inventory (  
  store_id         BIGINT NOT NULL REFERENCES stores(store_id),  
  product_id       BIGINT NOT NULL REFERENCES products(product_id),  
  quantity_on_hand INTEGER NOT NULL DEFAULT 0 CHECK (quantity_on_hand >= 0),  
  reorder_threshold INTEGER NOT NULL DEFAULT 0 CHECK (reorder_threshold >= 0),  
  PRIMARY KEY (store_id, product_id)  
);
```

because only one current inventory record should exist for each store-product combination. Similarly, `vendor_products` uses `UNIQUE (vendor_id, product_id)` to prevent the same vendor-product relationship from being defined more than once.

4.4.2 Business-Logic Constraints

Business rules are further enforced through `NOT NULL`, `DEFAULT`, `UNIQUE`, and `CHECK` constraints. The `purchase_order_items` table provides a representative example:

```
CREATE TABLE purchase_order_items (  
  purchase_order_item_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  purchase_order_id      BIGINT NOT NULL REFERENCES purchase_orders(purchase_order_id) ON DELETE CASCADE,  
  vendor_product_id      BIGINT NOT NULL REFERENCES vendor_products(vendor_product_id),  
  quantity_ordered       INTEGER NOT NULL CHECK (quantity_ordered > 0),  
  quantity_received      INTEGER NOT NULL DEFAULT 0 CHECK (quantity_received >= 0),  
  quantity_damaged       INTEGER NOT NULL DEFAULT 0 CHECK (quantity_damaged >= 0),  
  quantity_missing       INTEGER NOT NULL DEFAULT 0 CHECK (quantity_missing >= 0),  
  unit_purchase_cost     NUMERIC(10,2) NOT NULL CHECK (unit_purchase_cost >= 0),  
  UNIQUE (purchase_order_id, vendor_product_id),  
  CHECK (quantity_received <= quantity_ordered),  
  CHECK (quantity_damaged <= quantity_received)  
);
```

These constraints translate purchasing rules directly into the database. `NOT NULL` ensures that required information cannot be omitted, while `DEFAULT 0` provides an appropriate initial value when no units have yet been received, damaged, or identified as missing. `UNIQUE (purchase_order_id, vendor_product_id)` prevents the same vendor-product from appearing twice within one purchase order. The `CHECK` constraints prevent invalid quantities and costs: ordered quantities must be positive, received, damaged, and missing quantities cannot be negative, purchase cost cannot be negative, received units cannot exceed ordered units, and damaged units cannot exceed units actually received.

`ON DELETE CASCADE` reflects the dependency between an order and its line items. A `purchase_order_items` has no independent meaning once its parent purchase order is deleted, so dependent items are removed with the parent rather than being left as orphan records.

The same constraint principles are applied to the remaining tables according to their business meaning. Date checks maintain valid sequences for shifts, time-off requests, and purchase orders; controlled-value checks standardize statuses, payment methods, inventory-adjustment types, and expense types; and financial checks prevent invalid prices, discounts, refunds, payments, and expenses.

4.4.3 Triggers and Cross-Table Integrity

Some business rules depend on multiple tables and cannot be enforced using row-level CHECK constraints. Three PostgreSQL triggers were therefore implemented.

Inventory adjustment trigger: `trg_apply_inventory_adjustment` updates the corresponding `store_inventory` balance whenever an inventory adjustment is inserted and rejects the transaction if the resulting inventory would become negative. This keeps the current inventory balance consistent with its adjustment history.

Sales inventory trigger: `trg_reduce_inventory_after_sale` automatically creates a negative `inventory` adjustment when a `sales_items` record is inserted. This keeps sales activity and inventory movements synchronized.

Purchase-order vendor validation trigger: `trg_validate_purchase_order_vendor` verifies that the `vendor_product` selected for a purchase-order item belongs to the same vendor as the parent purchase order. This prevents a cross-table inconsistency that cannot be detected by either foreign key independently.

Together, primary and foreign keys `preserve` entity relationships, constraints enforce record-level business rules, and triggers enforce rules that require coordination across multiple tables. All the sql code is attached in Appendix

5. ETL Process

5.1 ETL Overview

After the relational database was designed, an ETL pipeline was developed in Python to prepare and load operational data into PostgreSQL. Because ABC Foodmart's existing records are assumed to come from separate spreadsheets and paper-based processes, the pipeline represents how data from different operational areas can be standardized, validated, and integrated into the centralized relational database.

For this project, reproducible synthetic data was generated to represent ABC Foodmart's operations because real company records were not available. The pipeline covers all 19 tables and follows three main stages: extraction into CSV staging files, transformation and validation using Python and pandas, and loading into PostgreSQL in dependency order. A fixed random seed and separate domain-level seeds were used to make the generated dataset reproducible.

Selected code listings are included below to illustrate the main ETL process.

5.2 Data Generation and Staging

In the generation stage, synthetic operational records were generated for the 19 tables in the database. The dataset covers store operations including staffing, products, inventory, vendors, purchasing, deliveries, sales, returns, payments, and operating expenses. The generated records are fictional and are intended to simulate the types of data that ABC Foodmart would maintain in a real operating environment.

The default dataset covers November 1, 2025 through July 31, 2026 and generates 36,000 sales transactions and 80 employees. A fixed random seed was used to make the generation process reproducible, while separate seeds were assigned to staffing, sales, purchasing, inventory loss, and expenses so that changes in one area would not unintentionally alter another.

Listing codes below shows the main configuration used to control the generation period, dataset size, and random seeds. Separating the random streams by operational domain improves reproducibility because changes to the generation logic in one area do not automatically change records generated for another area.

```
RANDOM_SEED = 5310
DEFAULT_START_DATE = date(2025, 11, 1)
DEFAULT_END_DATE = date(2026, 7, 31)
DEFAULT_SALES_COUNT = 36_000
DEFAULT_EMPLOYEE_COUNT = 80
DOMAIN_SEEDS = {
    "staffing": 531_001,
    "sales": 531_002,
    "purchasing": 531_003,
    "inventory_loss": 531_004,
    "expenses": 531_005,
}
```

The generated tables were not treated as independent random datasets. Related operational records were generated with business dependencies. For example, sales demand was aggregated by store, week, and product and used to determine replenishment requirements. Purchase quantities considered projected inventory, reorder thresholds, minimum order quantities, and a safety allowance. This produced purchasing records that were logically connected to simulated demand rather than completely random orders.

5.3 Cleaning and Standardization

During this stage, the staging files were read into pandas DataFrames and standardized before loading. Column names were converted to a consistent lowercase underscore format, text values were trimmed, and common representations of missing values such as "N/A", "null", and "unknown" were converted to null values. Monetary values were handled at two-decimal precision, while date and timestamp fields were converted to appropriate data types before validation.

Listing codes illustrates the main transformation process. Each CSV file is first read as text so that the pipeline can control type conversion explicitly. Column names and text values are then standardized before table-specific integer, numeric, date, timestamp, and Boolean conversions are applied. Malformed nonblank date or timestamp values cause the pipeline to raise an error rather than silently loading invalid data.

```
def transform_table(table_name: str, path: Path) -> pd.DataFrame:
    if not path.exists():
        raise FileNotFoundError(f"Missing staging file: {path}")
    raw_df = pd.read_csv(path, dtype=str)
    raw_df.columns = [
        standardize_column_name(column) for column in raw_df.columns
    ]
    df = raw_df.copy()
    # Trim text and convert common missing markers to NULL.
    for column in df.columns:
        df[column] = df[column].map(normalize_text)
    for column in INTEGER_COLUMNS.get(table_name, []):
        if column in df.columns:
            df[column] = pd.to_numeric(
                df[column], errors="raise"
            ).astype("int64")
    for column in NUMERIC_COLUMNS.get(table_name, []):
        if column in df.columns:
            df[column] = df[column].map(
                lambda value: None if value is None else money(value)
            )
```

The transformation stage also checks primary-key fields before loading. Null primary keys and duplicated primary-key combinations are rejected, preventing duplicated source records from being silently removed or transferred into the database.

5.4 Business-Rule Validation

Validation was performed before data was loaded into PostgreSQL. In addition to basic type and null checks, the pipeline tested cross-table business rules. One important example was inventory validation. Inventory movements and sales were replayed chronologically to verify that inventory never became negative at any point in the simulated history. This is stronger than checking only the final inventory balance because a positive ending balance could otherwise hide an earlier stock shortage.

Validation Area	Example Rule
Inventory	Stock cannot become negative after adjustments or sales
Purchasing	PO vendor must match the vendor associated with the product
Staffing	Employee shifts cannot conflict with approved time off
Staffing	Completed shifts must have valid actual working times
Returns	Return processor must be working at the correct store and time
Sales/Returns	Transaction refund amount must agree with detailed return records
Referential consistency	Related IDs must exist in their parent tables

If any business-rule validation fails, the pipeline raises an error.

5.5 PostgreSQL Loading Strategy

After cleaning and validation, the data was loaded into PostgreSQL. The loading sequence follows table dependencies created by foreign-key relationships. Parent and reference tables are loaded before dependent transaction tables. For example, stores and job roles must exist before employees, vendors and products must exist before vendor-product relationships, and sales transactions must exist before sales items and returns.

Also, because the database triggers described in Section 4 remain active during loading, the ETL process was designed to avoid duplicating trigger-generated records. When sales_items are inserted, the sales trigger automatically creates the corresponding inventory reduction. Therefore, the ETL process does not separately insert sale-related inventory adjustments.

5.6 Transaction Safety and Post-Load Validation

The complete database load is executed as a transaction. After all records are inserted, identity sequences are reset to the current maximum IDs and the loaded database is validated against the staging data. If all checks pass, the transaction is committed. If any exception or validation failure occurs, the transaction is rolled back so that the database is not left in a partially loaded state.

Post-load checks include table row counts, negative inventory, consistency of automatically generated sale adjustments, and consistency between analytical views and their underlying tables. This provides a final verification that both the ETL process and the database objects behave as expected after loading.

6. Analytical Procedures and Business Insights

Following database design and ETL, ten SQL-based analytical procedures were developed to transform operational records into information that supports store management decisions. The analyses use joins, conditional calculations, aggregations, ranking, and time-based transformations across sales, products, inventory, staffing, and expenses. Key results are also connected to the Metabase dashboard for interactive filtering by store and date.

6.1 Analysis 1 — Store Sales Performance

How does sales performance differ across stores?

```

1 - WITH sales AS (
2   SELECT DATE_TRUNC('month', st.sale_time)::date AS month,
3   SUM(CASE WHEN st.total_amount > 0 THEN
4     st.subtotal_amount - st.discount_amount
5     - ROUND(st.refund_amount * (st.subtotal_amount - st.discount_amount) / st.total_amount, 2)
6   ELSE 0 END) AS net_sales
7   FROM abc_foodmart.sales_transactions st
8   JOIN abc_foodmart.stores s ON s.store_id = st.store_id
9   WHERE st.sale_status <> 'voided'
10  [[AND st.sale_time::date >= {{start_date}}]]
11  [[AND st.sale_time::date <= {{end_date}}]]
12  [[AND {{store_name}}]]
13  GROUP BY 1
14 - ), expenses AS (
15   SELECT DATE_TRUNC('month', oe.expense_date)::date AS month,
16   SUM(oe.amount) AS operating_expenses
17   FROM abc_foodmart.operating_expenses oe
18   JOIN abc_foodmart.stores s ON s.store_id = oe.store_id
19   WHERE 1=1
20   [[AND oe.expense_date::date >= {{start_date}}]]
21   [[AND oe.expense_date::date <= {{end_date}}]]
22   [[AND {{store_name}}]]
23   GROUP BY 1
24 - )
25   SELECT COALESCE(sales.month, expenses.month) AS month,
26   ROUND(COALESCE(sales.net_sales, 0), 2) AS net_sales,
27   ROUND(COALESCE(expenses.operating_expenses, 0), 2) AS operating_expenses
28   FROM sales FULL OUTER JOIN expenses ON expenses.month = sales.month
29   ORDER BY 1;

```

This provides a starting point for investigating whether weaker store performance is associated with products, inventory, staffing, or other operating conditions.

6.2 Analysis 2 — Sales Trends and Timing

When does ABC Foodmart experience higher or lower sales demand?

```

1 v SELECT EXTRACT(HOUR FROM st.sale_time)::int sale_hour,
2     ROUND(SUM(st.total_amount-st.refund_amount),2) net_sales
3 FROM abc_foodmart.sales_transactions st
4 JOIN abc_foodmart.stores s ON s.store_id=st.store_id
5 WHERE st.sale_status <> 'voided'
6 [[AND st.sale_time::date >= {{start_date}}]]
7 [[AND st.sale_time::date <= {{end_date}}]]
8 [[AND {{store_name}}]]
9 GROUP BY 1 ORDER BY 1;

```

This supports staffing, cashier allocation, replenishment timing, and promotional planning around periods of higher demand.

6.3 Analysis 3 — Product and Category Performance

Which products and categories contribute most to sales volume and revenue?

```

1 v WITH returns AS (
2     SELECT sale_item_id, SUM(refund_amount) AS refund_amount
3     FROM abc_foodmart.customer_returns
4     GROUP BY sale_item_id
5 ),
6 category_sales AS (
7     SELECT pc.category_name,
8     SUM(
9         si.quantity_sold * si.unit_retail_price
10        - si.item_discount
11        - COALESCE(r.refund_amount, 0)
12    ) AS net_merchandise_sales
13 FROM abc_foodmart.sales_transactions st
14 JOIN abc_foodmart.sales_items si ON si.sale_id = st.sale_id
15 JOIN abc_foodmart.products p ON p.product_id = si.product_id
16 JOIN abc_foodmart.product_categories pc ON pc.category_id = p.category_id
17 JOIN abc_foodmart.stores s ON s.store_id = st.store_id
18 LEFT JOIN returns r ON r.sale_item_id = si.sale_item_id
19 WHERE st.sale_status <> 'voided'
20 [[AND st.sale_time::date >= {{start_date}}]]
21 [[AND st.sale_time::date <= {{end_date}}]]
22 [[AND {{store_name}}]]
23 GROUP BY pc.category_name
24 )
25 SELECT category_name,
26     ROUND(net_merchandise_sales, 2) AS net_merchandise_sales,
27     ROUND(100.0 * net_merchandise_sales / NULLIF(SUM(net_merchandise_sales) OVER (), 0), 1) AS sales_share_pct
28 FROM category_sales
29 ORDER BY net_merchandise_sales DESC;

```

This helps management prioritize products and categories for inventory allocation and purchasing based on both volume and financial contribution.

6.4 Analysis 4 — Product Return Analysis

Which products and categories have unusually high return rates?

```

1 v WITH returns AS (SELECT sale_item_id,SUM(quantity_returned) returned FROM abc_foodmart.customer_returns GROUP BY 1), product_stats AS (
2 SELECT p.product_name,SUM(si.quantity_sold) units_sold,COALESCE(SUM(r.returned),0) units_returned
3 FROM abc_foodmart.sales_transactions st
4 JOIN abc_foodmart.sales_items si ON si.sale_id=st.sale_id
5 JOIN abc_foodmart.products p ON p.product_id=si.product_id
6 JOIN abc_foodmart.stores s ON s.store_id=st.store_id
7 LEFT JOIN returns r ON r.sale_item_id=si.sale_item_id
8 WHERE st.sale_status <> 'voided'
9 [[AND st.sale_time::date >= {{start_date}}]]
10 [[AND st.sale_time::date <= {{end_date}}]]
11 [[AND {{store_name}}]]
12 GROUP BY p.product_name
13 )
14 SELECT product_name,units_sold,units_returned,ROUND(100.0*units_returned/NULLIF(units_sold,0),2) return_rate_pct
15 FROM product_stats WHERE units_sold>=100 ORDER BY return_rate_pct DESC,units_sold DESC LIMIT 10;

```

This provides an exception-identification mechanism for products that may require further investigation into quality, storage, or store execution.

6.5 Analysis 5 — Reorder Priority Analysis

Which products are closest to their reorder thresholds?

```

1 v SELECT s.store_name,
2         p.product_name,
3         si.quantity_on_hand,
4         si.reorder_threshold,
5         si.quantity_on_hand - si.reorder_threshold AS units_above_reorder
6 FROM abc_foodmart.store_inventory si
7 JOIN abc_foodmart.stores s ON s.store_id = si.store_id
8 JOIN abc_foodmart.products p ON p.product_id = si.product_id
9 WHERE 1=1
10 [[AND {{store_name}}]]
11 ORDER BY units_above_reorder, s.store_name, p.product_name
12 LIMIT 10;

```

This converts inventory balances into actionable replenishment priorities and helps reduce future stockout risk.

6.6 Analysis 6 — Slow-Moving Inventory Analysis

Which products may represent excess or slow-moving inventory?

```

1 v WITH bounds AS (
2 SELECT COALESCE(
3     MAX(st.sale_time::date) FILTER (
4         WHERE st.sale_status <> 'voided'
5         [[AND st.sale_time::date <= {{end_date}}]]
6     ),
7     MAX(st.sale_time::date) FILTER (WHERE st.sale_status <> 'voided')
8 ) AS anchor_date
9 FROM abc_foodmart.sales_transactions st
10 ),
11 returns_by_item AS (
12 SELECT cr.sale_item_id, SUM(cr.quantity_returned) AS returned_units
13 FROM abc_foodmart.customer_returns cr
14 CROSS JOIN bounds b
15 WHERE cr.return_time::date <= b.anchor_date
16 GROUP BY cr.sale_item_id
17 ),
18 sales_30d AS (
19 SELECT st.store_id, si.product_id,
20        SUM(GREATEST(si.quantity_sold - COALESCE(r.returned_units, 0), 0)) AS net_units_sold_30d
21 FROM abc_foodmart.sales_transactions st
22 JOIN abc_foodmart.sales_items si ON si.sale_id = st.sale_id
23 LEFT JOIN returns_by_item r ON r.sale_item_id = si.sale_item_id
24 CROSS JOIN bounds b
25 WHERE st.sale_status <> 'voided'
26       AND st.sale_time::date BETWEEN b.anchor_date - 29 AND b.anchor_date
27 GROUP BY st.store_id, si.product_id
28 ),
29 adjustments_after AS (
30 SELECT ia.store_id, ia.product_id, SUM(ia.quantity_delta) AS quantity_delta_after
31 FROM abc_foodmart.inventory_adjustments ia
32 CROSS JOIN bounds b
33 WHERE ia.adjustment_time::date > b.anchor_date
34 GROUP BY ia.store_id, ia.product_id
35 );

```

```

36 < base AS (
37     SELECT pc.category_id,
38            pc.category_name,
39            GREATEST(si.quantity_on_hand - COALESCE(aa.quantity_delta_after, 0), 0)::numeric AS inventory_units,
40            COALESCE(sa.net_units_sold_30d, 0)::numeric AS net_units_sold_30d
41     FROM abc_foodmart.store_inventory si
42     JOIN abc_foodmart.stores s ON s.store_id = si.store_id
43     JOIN abc_foodmart.products p ON p.product_id = si.product_id
44     JOIN abc_foodmart.product_categories pc ON pc.category_id = p.category_id
45     LEFT JOIN sales_30d sa ON sa.store_id = si.store_id AND sa.product_id = si.product_id
46     LEFT JOIN adjustments_after aa ON aa.store_id = si.store_id AND aa.product_id = si.product_id
47     WHERE 1=1 [[AND {{store_name}}]]
48 )
49 < metrics AS (
50     SELECT *,
51            CASE
52              WHEN net_units_sold_30d = 0 AND inventory_units > 0 THEN 'NO SALES'
53              WHEN inventory_units / NULLIF(net_units_sold_30d / 30.0, 0) > 30 THEN 'HIGH RISK'
54              WHEN inventory_units / NULLIF(net_units_sold_30d / 30.0, 0) >= 10 THEN 'WATCH'
55              ELSE 'NORMAL'
56            END AS risk_status
57     FROM base
58 )
59 < SELECT pc.category_name,
60        COALESCE(SUM(m.inventory_units) FILTER (WHERE m.risk_status <> 'NORMAL'), 0)::int AS slow_inventory_units
61     FROM abc_foodmart.product_categories pc
62     LEFT JOIN metrics m ON m.category_id = pc.category_id
63     GROUP BY pc.category_id, pc.category_name
64     ORDER BY slow_inventory_units DESC, pc.category_name;

```

Slow-moving inventory is identified by comparing inventory levels with net sales during the most recent 30 days. Net Units Sold is calculated after accounting for returns, and Average Daily Net Sales is calculated over the 30-day period. The analysis then estimates how many days the current inventory would last at the recent sales rate. Products with no sales but positive inventory are classified as No Sales, those with more than 30 days of inventory as High Risk, those with 10–30 days as Watch, and the remaining products as Normal.

6.7 Analysis 7 — Inventory Loss Analysis

Where are non-sales inventory losses occurring?

```

1 < WITH loss AS (
2     SELECT p.category_id, SUM(ABS(ia.quantity_delta)) loss_units
3     FROM abc_foodmart.inventory_adjustments ia
4     JOIN abc_foodmart.products p ON p.product_id=ia.product_id
5     JOIN abc_foodmart.stores s ON s.store_id=ia.store_id
6     WHERE ia.adjustment_type IN ('damage', 'expired')
7     [[AND ia.adjustment_time::date >= {{start_date}}]]
8     [[AND ia.adjustment_time::date <= {{end_date}}]]
9     [[AND {{store_name}}]]
10    GROUP BY p.category_id
11 )
12 < SELECT pc.category_name, COALESCE(loss.loss_units, 0) loss_units
13     FROM abc_foodmart.product_categories pc
14     LEFT JOIN loss ON loss.category_id=pc.category_id
15     ORDER BY loss_units DESC, pc.category_name;

```

This helps management distinguish normal stock movements from actual losses and focus investigation on categories with greater shrinkage or spoilage risk.

6.8 Analysis 8 — Labor Utilization Analysis

How closely do actual labor hours match scheduled labor hours?


```

1 v SELECT s.store_name,
2       ROUND(SUM(EXTRACT(EPOCH FROM (sh.scheduled_end - sh.scheduled_start))/3600.0),1) AS scheduled_hours,
3 v     ROUND(SUM(CASE WHEN sh.shift_status = 'completed' AND sh.actual_start IS NOT NULL AND sh.actual_end IS NOT NULL
4               THEN EXTRACT(EPOCH FROM (sh.actual_end - sh.actual_start))/3600.0 ELSE 0 END),1) AS actual_hours
5 FROM abc_foodmart.staff_shifts sh
6 JOIN abc_foodmart.stores s ON s.store_id = sh.store_id
7 WHERE 1=1
8 [[AND sh.scheduled_start::date >= {{start_date}}]]
9 [[AND sh.scheduled_start::date <= {{end_date}}]]
10 [[AND {{store_name}}]]
11 GROUP BY s.store_name
12 ORDER BY s.store_name;

```

This helps managers evaluate whether staffing plans are being executed as expected and identify where labor resources are concentrated.

6.9 Analysis 9 — Attendance Exception Analysis

Which employees show attendance exceptions requiring management attention?

```

1 v SELECT e.first_name || ' ' || e.last_name AS employee_name,
2       COUNT(*) FILTER (WHERE sh.shift_status = 'missed') AS absent_shifts,
3       COUNT(*) FILTER (
4         WHERE sh.shift_status = 'completed'
5         AND sh.actual_start > sh.scheduled_start + INTERVAL '5 minutes'
6       ) AS late_arrivals
7 FROM abc_foodmart.staff_shifts sh
8 JOIN abc_foodmart.employees e ON e.employee_id = sh.employee_id
9 JOIN abc_foodmart.stores s ON s.store_id = sh.store_id
10 WHERE 1=1
11 [[AND sh.scheduled_start::date >= {{start_date}}]]
12 [[AND sh.scheduled_start::date <= {{end_date}}]]
13 [[AND {{store_name}}]]
14 GROUP BY e.employee_id, e.first_name, e.last_name
15 HAVING COUNT(*) FILTER (WHERE sh.shift_status = 'missed') > 0
16       OR COUNT(*) FILTER (
17         WHERE sh.shift_status = 'completed'
18         AND sh.actual_start > sh.scheduled_start + INTERVAL '5 minutes'
19       ) > 0
20 ORDER BY absent_shifts DESC, late_arrivals DESC, employee_name
21 LIMIT 10;

```

Converts large volumes of shift records into exception-focused information so managers can prioritize cases requiring review.

6.10 Analysis 10 — Operating Expense Structure

What are the major sources of operating costs at ABC Foodmart?

```

1 v SELECT oe.expense_type,
2       ROUND(SUM(oe.amount),2) AS operating_expenses
3 FROM abc_foodmart.operating_expenses oe
4 JOIN abc_foodmart.stores s ON s.store_id = oe.store_id
5 WHERE 1=1
6 [[AND oe.expense_date::date >= {{start_date}}]]
7 [[AND oe.expense_date::date <= {{end_date}}]]
8 [[AND {{store_name}}]]
9 GROUP BY oe.expense_type
10 ORDER BY operating_expenses DESC;

```

Shows that labor and rent dominate the cost structure, making labor utilization a particularly important area for cost-efficiency analysis.

6.11 Overall Business Insights

The ten analyses form a connected view of store operations rather than isolated performance measures. Sales timing can inform staffing decisions; product performance can guide replenishment; reorder and slow-moving inventory analyses identify opposite inventory risks; inventory-loss analysis highlights potential shrinkage or spoilage; and labor utilization directly connects staffing decisions with the largest component of operating expenses.

Together, these procedures demonstrate the value of the relational database by linking sales, products, inventory, employees, and expenses through shared store, product, employee, transaction, and date relationships. Metabase presents the SQL results as interactive KPIs, trends, rankings, and detail tables, allowing managers to move from overall performance indicators to specific operational issues requiring action.

7. User Interaction and System Architecture

The system provides different levels of interaction based on user roles. Analysts can directly access PostgreSQL using raw SQL or Python/R connectors, allowing them to run and modify the analytical procedures in Section 6 and perform detailed drill-down analysis. Using programming languages also makes repeated database operations more efficient and reproducible.

Managers and executives interact with the database primarily through Metabase. Analytical results are presented as KPIs, charts, rankings, and detailed tables, with store and date filters for interactive analysis. This allows non-technical users to review business performance and identify issues without writing SQL or directly accessing the underlying database.

7.1 Performance, Redundancy and Deployment

During the database design process, our strategy was to reduce data redundancy through 3NF and improve performance through indexing, query optimization, and potential future use of views. For dashboard queries that are executed repeatedly, regular views or materialized views could also be considered in the future to reduce repeated computation. It is important to note that these mechanisms were not actually

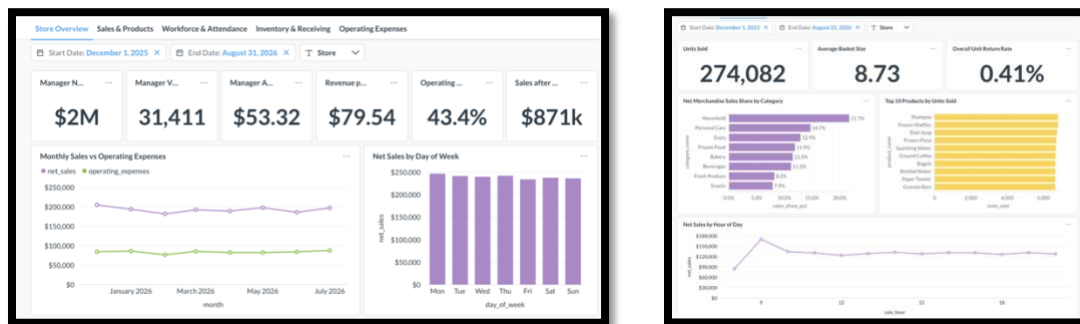
implemented in this project and are better considered as future extensions once the system is deployed for real business operations.

For deployment, we recommend that ABC Foodmart use a cloud-based solution. ABC Foodmart is expanding from two stores to five locations, making cloud deployment more suitable for a multi-store environment. In addition, ABC Foodmart requires a real-time dashboard, which further increases the need for a cloud-based system, since the database and dashboard should not depend on someone manually refreshing or maintaining the data at all times.

7.2 Metabase Dashboards

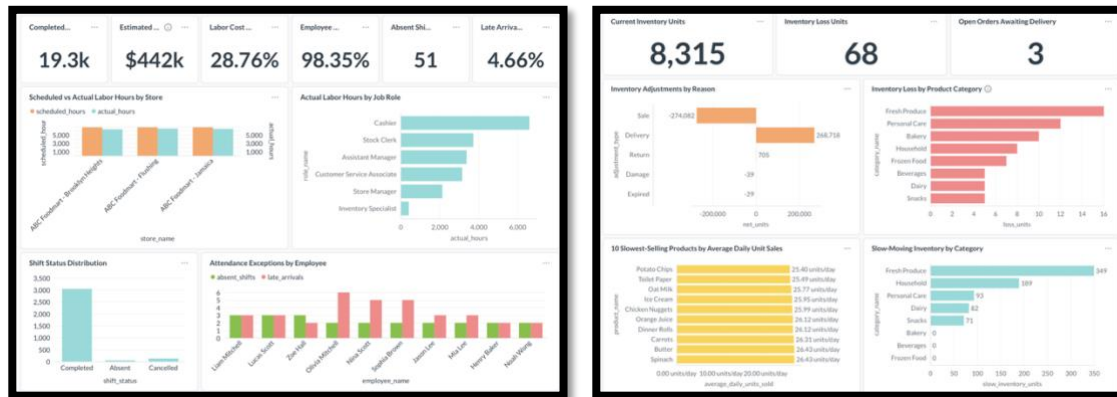
The PostgreSQL database was connected to Metabase to provide an interactive reporting interface for managers and executives. The dashboard is organized into five business areas, with store and date filters allowing users to dynamically review different locations and periods.

The Store Overview provides a high-level view of business performance through key sales and operating KPIs, together with sales and expense trends. It allows managers to quickly assess overall store performance before investigating specific business areas.

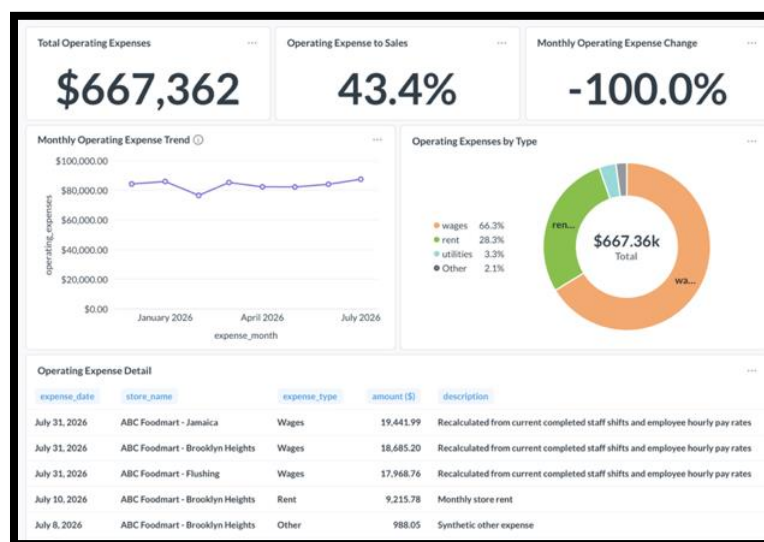


The Sales & Products dashboard presents sales volume, basket size, category contribution, product rankings, return performance, and sales timing. It helps managers identify major sales drivers, high-return products, and periods of stronger customer demand.

The Workforce & Attendance dashboard compares scheduled and actual labor hours and summarizes attendance performance and employee exceptions. It helps managers identify staffing gaps and employees or job roles requiring further attention.



The Inventory & Receiving dashboard presents current inventory, inventory losses, outstanding orders, slow-moving stock, and reorder information. It supports replenishment decisions while also highlighting excess inventory and potential inventory losses.



The Operating Expenses dashboard presents total expenses, the expense-to-sales ratio, monthly cost trends, and the distribution of expenses by type. It helps management identify major cost drivers and monitor changes in operating costs over time.

Overall, the dashboards convert the SQL analytical procedures into an accessible management interface. Non-technical users can monitor KPIs, apply store and date filters, identify exceptions, and drill down into detailed information. This improves the accessibility and efficiency of data-driven decision-making.

8. Conclusion

This project developed a centralized database and analytics solution for ABC Foodmart to replace fragmented operational records and support future expansion. The PostgreSQL relational database integrates 19 tables across sales, inventory, purchasing, workforce, and operating expenses, improving data consistency and enabling cross-functional analysis. A Python ETL pipeline was used to prepare, validate, and reliably load data, while SQL analytical procedures generated insights into sales patterns, product performance, inventory management, labor utilization, attendance, and operating costs. These results were presented through interactive Metabase dashboards, allowing managers to monitor KPIs, apply store and date filters, and investigate operational issues without writing SQL. Overall, the integration of PostgreSQL, Python ETL, SQL analytics, and Metabase provides ABC Foodmart with a more consistent, reproducible, and scalable system for data-driven operational and strategic decision-making.

Appendix

1. GitHub Repository:
https://github.com/celinewang020805-afk/CU_APAN5310_Final_Project
2. Full Dataset – Github Version:
https://github.com/celinewang020805-afk/CU_APAN5310_Final_Project/tree/070627bde08b3dbd847a2fe7dcc8c2eb2cf0edc9/02_Data
3. Full Dataset - Google Drive Version:
https://drive.google.com/drive/folders/1XZ3Zo5_3Stk0EV1tsyX9YEjXcNXMNlv?usp=share_link
4. Database Schema / SQL Code:
https://github.com/celinewang020805-afk/CU_APAN5310_Final_Project/blob/5b4653a3d41856daa96d80245e6d45fcbbd01674/01_Database/checkpoint3_schema_candidate.sql
5. ER Diagram – PostgreSQL Version:
https://github.com/celinewang020805-afk/CU_APAN5310_Final_Project/blob/5b4653a3d41856daa96d80245e6d45fcbbd01674/01_Database/ER_Diagram.png
6. ER Diagram – LucidChart Version:
https://lucid.app/lucidchart/df33fdb0-a32d-48d7-8c5f-6392c9f83a02/edit?viewport_loc=-2235%2C-2794%2C4583%2C2779%2C0_0&invitationId=inv_0d4f1b21-aa1d-4f5d-818f-13758cd7b2bc
7. ETL / Data Generation Code:
https://github.com/celinewang020805-afk/CU_APAN5310_Final_Project/blob/5b4653a3d41856daa96d80245e6d45fcbbd01674/03_ETL_Data_Generation/ABC_Foodmart_Data_Generation.ipynb
8. Metabase Dashboard:
<http://82.157.4.243:3000>
(Username:3441136350@qq.com Password: Ty197399&)
9. Final Slides:
https://github.com/celinewang020805-afk/CU_APAN5310_Final_Project/blob/5b4653a3d41856daa96d80245e6d45fcbbd01674/05_Final_Submission/ABC_Foodmart_Pre.pdf